

CS201 Fall 2025 — Lecture Notes

Clarissa Littler

Fall 2025

Preface

This document is a running narrative of what we covered in each class meeting this term. Each section pairs the agenda for the day with the little code samples we actually typed up together, so if you missed a lecture or want to go back and re-read, the whole arc of a day should be here in one place. The source files themselves live in the `lectureN/` subdirectories of the class repo, and all of the snippets quoted below are verbatim from those files.

The numbering reflects the lecture sequence as it actually ran — some numbers are skipped (those days were holidays, midterms, or discussion-only sessions). After the numbered sequence I’ve included the file-I/O day, the sockets day, and a short appendix on the pointer-pyrotechnics examples.

Contents

1	Lecture 1 — Welcome, C vs. C++, Bit-Munging, and Two’s Complement	1
2	Lecture 2 — Floating Point, IEEE-754, malloc on Structs	3
3	Lecture 3 — Hex, Pointer Arithmetic, Casting, and Endianness	6
4	Lecture 5 — Your First Assembly: exit, add, and Looking at What gcc -S Emits	8
5	Lecture 6 — cmp, jmp, Loops, and Globals	10
6	Lecture 7 — Globals, lea, Arrays, read/write, and Functions	12
7	Lecture 8 — The Stack, Local Variables, readInt/writeInt, and Recursion	16
8	Lecture 9 — Processes and the First Look at Threads	20
9	Lecture 11 — Function Pointers, Threads in Earnest, and Mutexes	23
10	File I/O — Files, File Descriptors, argv, and a Line Editor	26
11	Lecture 13 — Signals and Signal Handlers	31
12	Lecture 14 — Threads in Earnest, and Reworking the Incremental Game	35
13	Sockets — Networks, the TCP/IP Stack, and Three Echo Servers	38

1 Lecture 1 — Welcome, C vs. C++, Bit-Munging, and Two's Complement

This was an orientation day plus the first run at the parts of C that trip people up coming from C++, plus our opening foray into bitwise operations and integer representation.

Logistics

The class is my own personal admixture of four things:

- C
- x86-64 assembly
- Concurrency
- Operating systems theory and practice

Our official hardware is `cslinux.pcc.edu`; if you can't log in yet, the stopgap is `onlinegdb.com`. The repo is where the textbook lives, where these Agenda files come from, and where every bit of code we write together gets dropped.

C: what's different from C++?

The smaller surprises:

- `struct` is its own keyword and you usually have to write `struct Foo` when referring to one.
- No native `bool` — nonzero is truthy, zero is falsy, and that's that.

The bigger surprises are the I/O functions: `printf` and `scanf` instead of streams. `firstC.c` is our first worked example, showing both `malloc` on an array and `printf` as the way you talk to the world:

```
#include "stdio.h"
#include "stdbool.h"
#include "stdlib.h"

struct Rect {
    int width;
    int height;
};

int main(){
    int* arr1 = malloc(100*sizeof(int));

    for(int i=0; i<100; i++){
        *(arr1 + i) = i*i;
        printf("The %d th element is %d\n",i,arr1[i]);
    }

    free(arr1);
}
```

```
    return 0;
}
```

Two things to soak in here. First, `*(arr1 + i)` and `arr1[i]` are literally the same thing — indexing is just pointer arithmetic in disguise. We’ll lean on that all term. Second, `malloc` returns a pointer and needs a size in bytes; unlike `new T` in C++, it doesn’t know your type.

(Side note that will come back to bite us in the audit: I keep writing `#include "stdio.h"` with quotes instead of `<stdio.h>`. It works on our setup because the system headers end up resolved either way, but the angle-bracket form is the correct convention for system headers — I’ll start using it more strictly later.)

Bitwise operations

We went through the usual cast: `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), and the shifts `<<` and `>>`. The classic XOR trick is “XOR-ing a bit with 1 flips it, XOR-ing with 0 leaves it alone,” which is the entire content of `toggleBit` in `bitmunger1.c`:

```
void printBits(int num1){
    printf("%d in bits is: ",num1);
    for(int i=31; i>=0;i--){
        printf("%d", (num1>>i)&1);
    }
    printf("\n");
}

void toggleBit(int* num,int place){
    *num = (*num) ^ (1 << place);
}
```

The `printBits` function is itself a nice little exercise in bit-twiddling: shift the bit you want down to position 0, then mask with `& 1` to get just that bit. The driver loops, prints the integer in both signed and unsigned interpretations, and lets you flip bits interactively until you type -1.

Two’s complement, derived

A plain n -bit unsigned integer encodes

$$\sum_{i=0}^{n-1} b_i \cdot 2^i,$$

so all-ones in 32 bits is $2^{32} - 1$. In two’s complement, the top bit’s weight goes negative:

$$-b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i.$$

That’s the whole magic. The largest signed 32-bit value is $2^{31} - 1$, the smallest is -2^{31} , and all-ones is always -1 because $-2^{n-1} + (2^{n-1} - 1) = -1$.

2 Lecture 2 — Floating Point, IEEE-754, malloc on Structs

Today we changed our bit-munger to look at floats, talked through the IEEE-754 standard, then pivoted to `malloc` on structs and pointers in C.

Floats and the bit-munger

bitmunger2.c is structurally the same as last time's `bitmunger1.c`, but it reinterprets the bits of a float via a pointer cast. The little spaces it prints (between bits 31/30 and 23/22) are previewing the IEEE-754 sign / exponent / fraction split:

```
void printBits(int* num1){
    printf("in bits is: ");
    for(int i=31; i>=0;i--){
        printf("%d",(*num1>>i)&1);
        if(i == 31 || i == 23){
            printf(" ");
        }
    }
    printf("\n");
}

int main(){
    float num1;
    printf("Choose a starting number: ");
    scanf("%f",&num1);
    int choice;

    do {
        printf("The number %f ",num1);
        printBits((int*)&num1);
        printf("Enter a bit to flip: ");
        scanf("%d",&choice);
        toggleBit((int*)&num1,choice);
    }while (choice != -1);
}
```

The `(int*)&num1` cast is the load-bearing trick: take the address of the float, lie to the compiler about its type, and now we can twiddle bits with integer operations and watch the floating-point value change.

IEEE-754, formally

A 32-bit float has:

- 1 sign bit
- 8 exponent bits (stored with bias 127)
- 23 fraction bits

For normal values, the value encoded is

$$(-1)^s \cdot 2^{e-127} \cdot \left(1 + \sum_{k=1}^{23} f_k \cdot 2^{-k}\right).$$

Edge cases:

- Exponent all 1s, fraction zero $\Rightarrow \pm\infty$.

- Exponent all 1s, fraction nonzero $\Rightarrow$ NaN.
- Exponent all 0s $\Rightarrow$ subnormals: the implicit leading 1 is dropped and the exponent is fixed at 2^{-126} , so we can gracefully approach 0.

Where floats lie to you

floatround.c is the gotcha:

```
double num = 0.00001;
double acc = 0;

for(int i = 0; i < 100000; i++){
    acc = acc + num;
}

if(acc != 1){
    printf("UH OH %f is not equal to 1\n", acc);
}
if((acc < 1 + 0.000001) && acc > (1 - 0.000001)){
    printf("But it's close enough!\n");
}
```

We accumulate a quantity that, in math-land, should be exactly 1. It isn't, because 0.00001 has no exact binary representation, and we're piling up the rounding error 100,000 times. The “but it's close enough” check is the practical answer: never compare floats with ==; always use a tolerance.

The deeper reason this is unavoidable: the rationals are countably infinite, the reals are uncountable. There is no way to cover the reals with finitely many bits; we're forced to scatter “breadcrumbs” across the number line and round everything to its nearest breadcrumb.

malloc on structs and pointer sizes

malloc2.c extends malloc from arrays to structs:

```
struct Rect {
    int width;
    int height;
};

int main(){
    struct Rect* r = malloc(sizeof(struct Rect));

    r->width = 10; // (*r).width
    r->height = 20;

    printf("%d\n", r->width * r->height);

    free(r);
    return 0;
}
```

The arrow `->` is shorthand for “dereference, then access a field.” `r->width` is the same as `(*r).width` — the parens matter because `*` has lower precedence than `..`

malloc3.c shows that `sizeof` on a struct gives back the total bytes the compiler reserved (which can include padding):

```
struct Goofus {
    int arr[100];
};

int main(){
    struct Goofus* g = malloc(sizeof(struct Goofus));
    printf("A goofus is %ld bytes big\n",sizeof(struct Goofus));
    free(g);
    return 0;
}
```

And `pointerSize.c` drives home that all pointers are the same size — 8 bytes on a 64-bit machine — regardless of what they point to:

```
int num1;
float num2;
long num3;
double num4;
printf("The size of an int* is: %ld\n",sizeof(&num1));
printf("The size of a long* is: %ld\n",sizeof(&num3));
printf("The size of a float* is: %ld\n",sizeof(&num2));
printf("The size of a double* is: %ld\n",sizeof(&num4));
```

Stacks (the data structure)

The agenda also has a sketch for a stack data structure exercise. We didn't fully implement it in class, but the shape is:

```
struct Stack {
    // an array, plus a current length
    // pick a finite max size
};

void push(struct Stack *s, int i);
int pop(struct Stack *s);
int length(struct Stack s);
void printStack(struct Stack s);
```

This will reappear later when we talk about the *call* stack in assembly, which is a real instance of this same data structure managed by the CPU.

3 Lecture 3 — Hex, Pointer Arithmetic, Casting, and Endianness

Today was about getting fluent with hex output, the actual mechanics of pointer arithmetic, casting between pointer types, and seeing endianness with our own eyes.

Why hex

`whyHex.c` is a trivial program that asks `printf` to show the same number three ways:

```
int num1 = 1234;
printf("As decimal: %d\n", num1);
printf("As hex: %x\n", num1);
printf("As octal: %o\n", num1);
```

The reason hex shows up everywhere — addresses, color codes, byte dumps — is that two hex digits are exactly one byte ($16 \cdot 16 = 256 = 2^8$). So 0xFF is one byte all-ones, which as unsigned is 255.

printHex.c prints the address of every element of a `char[10]`, which lets you see the addresses going up by exactly 1 each step (because `sizeof(char) == 1`).

Pointer arithmetic, made tangible

pointerArith.c shows the rule you have to internalize: pointer arithmetic scales with the *pointee* type:

```
int num1;
printf("The address of num1 is: %p\n", (void*)&num1);
printf("The address of &num1 + 1 is: %p\n", (void*)(&num1 + 1));
```

The two addresses printed differ by 4 (because `sizeof(int) == 4`), not by 1.

Casting and “zoom level”

byteMunging.c is a great little demo: cast the same array pointer to different types and watch +1 mean different things:

```
int arr[10];
for(int i = 0; i < 10; i++){
    arr[i] = i*i;
}

printf("If we don't cast at all arr+1 is: %p\n", (void*)(arr + 1));
printf("If we cast to long, arr+1 is: %p\n", (void*)((long*)arr + 1));
printf("If we cast to char, arr+1 is: %p\n", (void*)((char*)arr + 1));
```

As an `int*`, +1 jumps 4 bytes; as a `long*`, +1 jumps 8; as a `char*`, +1 jumps 1. The “zoom level” of a pointer is encoded entirely in the type.

Endianness

byteOrder.c actually peers into the bytes of an integer:

```
int num = 0x1a2b3c4d;

unsigned char* pty = (unsigned char*)&num;
for(int i = 0; i < 4; i++){
    printf("%x ", pty[i]);
}
printf("\n");
```

On our x86-64 boxes this prints `4d 3c 2b 1a` — the opposite order from how we wrote the literal. That’s little-endian: the least-significant byte is at the lowest address. The cast to `unsigned char*` is the trick: once we have a byte-pointer, pointer arithmetic moves us one byte at a time, so we can walk the actual storage.

There’s also `byteOrder.s`, which is what `gcc -S` emits for this C program. We didn’t pick through it line-by-line yet, but it’s there to look at — the `movzbl`, `leaq`, and `call printf@PLT` are all things we’ll unpack in detail when we get to assembly proper.

Two’s complement, redux

The agenda revisits two’s complement with worked examples for one byte:

- Unsigned: `0111 1111` = 127, `1000 0000` = 128, `1111 1111` = 255.
- Signed (two’s complement): `0111 1111` = 127, `1000 0000` = -128, `1111 1111` = -1.

And it shows the wraparound: `127 + 1` in 8 bits is `1000 0000`, which as unsigned is 128 and as signed is -128. That’s overflow, and it’s why $2^{31} - 1 + 1$ becomes -2^{31} on a 32-bit signed int.

The agenda also flags two real-world dates this matters for: Y2K (rolling 2-digit years from 99 to 00) and Y2038 (the moment 32-bit signed Unix timestamps overflow). Same arithmetic, different calendar.

4 Lecture 5 — Your First Assembly: `exit`, `add`, and Looking at What `gcc -S` Emits

(There was no Lecture 4 directory — we shifted scheduling.) Today was the big jump from C to assembly. We talked about what a processor actually *is*, then wrote the smallest possible assembly programs against the bare Linux syscall interface.

What is a processor?

- A CPU fetches instructions from memory and does what they say. Most instructions are one clock cycle (with a giant asterisk).
- All modern CPUs have a *clock* — an oscillator that drives signals to happen synchronously.
- The set of instructions a CPU understands is its *instruction set architecture* (ISA). Common ones: x86 (legacy 32-bit), x86-64 (modern 64-bit, what we’re using), ARM64 (Apple Silicon, phones), RISC-V (open hardware).
- Asking humans to write raw byte sequences is awful. So we invented *assembly languages* — human-readable shorthand for the instruction set, defined by an *assembler*. We’re using GNU `as`, which speaks AT&T syntax.

Pipeline: compiler → assembler → linker

- The *compiler* turns C into assembly.
- The *assembler* turns assembly into raw machine instructions in an object file.

- But an object file isn't yet executable — the *linker* pulls in library code and OS boilerplate to produce the actual binary.

You can stop after the compiler stage with `gcc -S file.c`, which spits out `file.s`. We'll use that all term to peek at what C compiles to.

Registers

- Registers are the CPU's tiny, blazing-fast scratchpad: 16 general-purpose registers in x86-64, each 64 bits.
- In AT&T syntax all register names start with a %.
- Some registers have conventional uses (we'll fill that table in next time); others (`%r8-%r15`) you can do whatever you want with.

The smallest program that doesn't crash

`smallest.s` is the smallest assembly program that runs without segfaulting:

```
.text
.global _start

_start:
    mov $60, %rax
    mov $-1, %rdi
    syscall
```

That's it. The Linux `exit` syscall is number 60, the exit status goes in `%rdi`, and `syscall` actually pulls the trigger. Run it and `echo $?` — you'll see 255 (which is how the shell reads `-1` as an unsigned byte).

A few notes:

- AT&T immediates are prefixed with `$`: `$60` is the constant 60, while `60` on its own would mean “the contents of memory address 60” (which would be a segfault).
- `.text` is the section where code lives.
- `.global _start` tells the linker that `_start` is the entry point. There's no `main` runtime here — the kernel jumps straight to `_start`.

add as a binary operation

`smaller.s` adds two numbers and feeds the sum back as the exit status:

```
.text
.global _start

_start:
    mov $1, %r8
    mov $2, %r9
    add %r8, %r9
    mov %r9, %rdi
    mov $60, %rax
```

The crucial syntax point that will bite you all term: **in AT&T syntax, add S, D computes $D = D + S$** . The destination is on the *right*. So `add %r8, %r9` leaves $1 + 2 = 3$ in `%r9`.

Skimming what `gcc -S` emits

For comparison, we wrote a tiny C program `adding.c` that adds two numbers via a function:

```
int adder(int n1, int n2){
    int res;
    res = n1 + n2;
    return res;
}

int main(){
    int n1 = 10;
    int n2 = 20;
    printf("%d\n", adder(n1, n2));
    return 0;
}
```

`gcc -S` produces `adding.s`, which is noisy with `.cfi_*` directives, frame setup, and PLT calls, but the actual work is recognizable:

```
adder:
    pushq %rbp
    movq %rsp, %rbp
    movl %edi, -20(%rbp)
    movl %esi, -24(%rbp)
    movl -20(%rbp), %edx
    movl -24(%rbp), %eax
    addl %edx, %eax
    movl %eax, -4(%rbp)
    movl -4(%rbp), %eax
    popq %rbp
    ret
```

`%edi` and `%esi` are the 32-bit halves of `%rdi` and `%rsi` — the first two integer arguments. The compiler spills them onto the stack (`-20(%rbp)`, `-24(%rbp)`), reloads them, adds, and returns the result in `%eax`. We'll formalize this calling convention next time.

`hello.c` is just `printf("Hello world\n");` — the placeholder for the C version of what we'll build in pure assembly later.

5 Lecture 6 — `cmp`, `jmp`, Loops, and Globals

With one assembly instruction (`add`) under our belts, we filled in control flow: `cmp`, the conditional jumps, and a real loop.

Quick recap

`smallredux.s` and `arith.s` are warm-ups from last time:

```
_start:
    mov $5,%r8
    add $10,%r8 # r8 = r8 + 10
    add %r8,%r8 # r8 = r8 + r8
    mov %r8,%rdi

    mov $60,%rax
    syscall
```

Exit status is 30. Adding a register to itself is the cheap way to double a value.

cmp and the conditional jumps

`cmp S, D` computes $D - S$ and sets flags but doesn't store the result:

- ZF (zero flag): was the result zero?
- SF (sign flag): was it negative?
- OF (overflow flag): did signed overflow happen?
- CF (carry flag): did unsigned carry happen?

The conditional jumps then read those flags. Critical AT&T gotcha: after `cmp %r8, %r9`, the jump `j1` means $\%r9 < \%r8$ (because the implicit comparison is $\%r9 - \%r8$). Trips me up every time.

`cmp1.s` branches on equality:

```
_start:
    mov $2,%r8
    mov $2,%r9
    cmp %r8,%r9
    je equal
    mov $0,%rdi # 0 if not equal
    jmp finish

equal:
    mov $1,%rdi # 1 if equal

finish:
    mov $60,%rax
    syscall
```

`cmp2.s` branches on `j1` (less-than) with two register operands; `cmp3.s` compares against an immediate:

```
    cmp $3,%r9 # compare %r9 against the immediate 3
    j1 less
```

A for-loop in assembly

for1.s sums 1 through 10:

```
_start:
    mov $1,%rcx
    mov $0,%r8

loop:
    cmp $11,%rcx
    jge finish
    add %rcx,%r8
    add $1,%rcx
    jmp loop

finish:
    mov %r8,%rdi
    mov $60,%rax
    syscall
```

`%rcx` is the conventional loop counter. The structure is “test before the body” — a while-style loop. Exit status is 55.

Globals and the wonder of `%rip`-relative addressing

for2.s introduces global variables in the `.data` section and walks them through the same loop:

```
.section .data
num: .quad 0
num2: .quad 0

.section .text
.global _start

_start:
    mov $1,%rcx

loop:
    cmp $11,%rcx
    jge finish
    add %rcx,num(%rip)
    addq $1,num2(%rip)
    add $1,%rcx
    jmp loop

finish:
    mov num(%rip),%rdi
    mov $60,%rax
    syscall
```

The mysterious `(%rip)` bit is RIP-relative addressing. Here’s the conceptual picture: `%rip` is the instruction pointer, the address of the instruction currently executing. The assembler doesn’t know where `num` will live at runtime (that depends on the loader and ASLR), but it *does* know how many

bytes `num` is from this very instruction. So `num(%rip)` is really shorthand for “take the runtime `%rip`, add this assembler-computed offset, that’s where `num` lives.”

This is the modern, position-independent way to refer to globals. We’ll keep using it.

6 Lecture 7 — Globals, `lea`, Arrays, read/write, and Functions

A dense one. We made global variables real, used `lea` to take addresses, walked an array four ways, did `read` and `write` both in C and in assembly, and wrote our first function.

Globals, properly

`global1.s` is the cleanest possible single-global program:

```
.section .data
num: .quad 10 # 64 bit integer (long)

.section .text
.global _start

_start:
    addq $10,num(%rip)
    movq num(%rip), %rdi
    movq $60, %rax
    syscall
```

A few things to notice:

- Globals declare a *size*, not a type. `.quad` is 8 bytes, `.long` is 4, `.word` is 2, `.byte` is 1.
- `addq` has a `q` suffix because we’re operating on 64-bit (quadword) data. The size suffixes (`b`, `w`, `l`, `q`) become important when memory operands don’t pin down the size.
- I write `.global _start` because the linker has to see that symbol from outside; I do *not* write `.global num` because nothing outside this file needs to refer to it.

`lea`: how pointers actually work

`lea` (“load effective address”) is the assembly equivalent of C’s `&`. `global2.s`:

```
_start:
    lea num(%rip),%r9
    # %r9 = &num
    addq $10,(%r9)
    # *(%r9) = *(%r9) + 10
    movq (%r9), %rdi
    movq $60, %rax
    syscall
```

The parens around a register — `(%r9)` — mean “dereference,” the same as the C `*` operator. So `addq $10, (%r9)` adds 10 to the 8 bytes at the address held in `%r9`. That maps to C as `*ptr = *ptr + 10`.

Arrays in assembly

Three files, one new addressing trick at a time.

`globalArray1.s` declares an array as a sequence of values under one label:

```
arr: .quad 1,2,3,4

_start:
    lea arr(%rip),%r9
    mov (%r9), %rdi # *arr -> 1
    mov $60, %rax
    syscall
```

That gives us element 0. To get other elements, `globalArray2.s` uses scale-index addressing:

```
    lea arr(%rip),%r9
    mov $1,%rcx
    mov (%r9,%rcx,8), %rdi
    # *(%r9 + 1*8)
```

The form (base, index, scale) computes `base + index*scale`, where the scale is 1, 2, 4, or 8. Since our elements are `.quad` (8 bytes), the scale is 8.

`globalArray3.s` sums a 5-element array with this pattern in a loop:

```
arr: .quad 1,2,3,4,5

_start:
    lea arr(%rip),%r8
    mov $0,%r9 # accumulator
    mov $0,%rcx # i = 0

loopStart:
    cmp $5,%rcx
    jge loopEnd
    add (%r8,%rcx,8),%r9
    add $1,%rcx
    jmp loopStart

loopEnd:
    mov %r9,%rdi
    mov $60,%rax
    syscall
```

Exit status is 15. This pattern — base in one register, index in another, scale equal to element size — is exactly how C array indexing compiles.

read and write, first in C

`readWrite1.c` uses the raw POSIX `write` syscall wrapper:

```
#include "stdio.h"
#include "unistd.h"
```

```
int main(){
    char hiThere[] = "Hey there, goofuses and goofuxen\n";
    write(1, hiThere, 33);
    return 0;
}
```

The arguments are (fd, buf, count), where 1 is stdout. `readWrite2.c` mirrors it for input:

```
char buff[30];
int charsRead = read(0,buff,30);
write(1,buff,charsRead);
```

0 is stdin. `read` returns the number of bytes it actually got, and we feed that into `write` so we don't echo back uninitialized garbage at the tail of the buffer.

read and write, in assembly

`readWriteASM1.s` writes a fixed string:

```
.section .data
hiThere: .ascii "Hey there, goofuses and goofuxen\n"

.section .text
.global _start

_start:
    mov $1,%rdi
    lea hiThere(%rip),%rsi
    mov $33,%rdx
    mov $1,%rax
    syscall

    mov $60,%rax
    mov $0,%rdi
    syscall
```

The Linux x86-64 syscall convention: syscall number in `%rax`, then arguments in `%rdi`, `%rsi`, `%rdx`, `%r10`, `%r8`, `%r9`. `write` is syscall 1, (fd, buf, count).

`readWriteASM2.s` chains `read` and `write` through a `.bss` buffer:

```
.section .bss
buff: .skip 100

.section .text
.global _start

_start:
    mov $0,%rdi
    lea buff(%rip),%rsi
    mov $100,%rdx
    mov $0,%rax
    syscall
    ## %rax now holds the number of bytes read
```

```

    mov $1,%rdi
    lea buff(%rip),%rsi
    mov %rax,%rdx
    mov $1,%rax
    syscall

    mov $60,%rax
    mov $0,%rdi
    syscall

```

`.bss` is zero-initialized scratch storage; `.skip 100` reserves 100 bytes. The clever line is `mov %rax, %rdx` — the `syscall` returns its byte-count in `%rax`, and we funnel that into `%rdx` as the count argument for `write`, so we only echo back what was actually typed.

Functions and the calling convention

The Linux x86-64 calling convention for user-space functions:

- Arguments in order: `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`.
- Return value in `%rax`.

(Syscalls are slightly different — the fourth argument is `%r10`, not `%rcx` — but for plain function calls, the list above is what you want.)

`firstFunc.s` is the first “real” function we wrote:

```

sillyAdd:
    mov $0,%rax
    add %rdi,%rax
    add %rsi,%rax
    ret

_start:
    mov $10,%rdi
    mov $20,%rsi
    call sillyAdd

    mov %rax,%rdi
    mov $60,%rax
    syscall

```

`call sillyAdd` pushes the address of the next instruction onto the stack and jumps; `ret` pops that address and jumps back. The stack carries the return address invisibly — that’s the whole magic trick.

7 Lecture 8 — The Stack, Local Variables, `readInt/writeInt`, and Recursion

Today we made the call stack real, wrote a function with locals by hand, linked in pre-written `readInt` and `writeInt` helpers, and warned about stack overflow via runaway recursion.

Logistics

Coding assignment 2 is due in two weeks. The next discussions are fluffier so people can really sit with assembly. Quizzes are unlimited attempts — if you don't like your score, just try again.

Local variables and the stack

Every process gets a chunk of address space laid out roughly:

- Stack at the top, growing *downward*.
- Heap (`malloc`-land) growing upward from somewhere lower.
- Data sections (`.data`, `.bss`, `.rodata`) below that.
- Executable instructions at the bottom.

Local variables live on the stack. Each function call gets a *stack frame* of contiguous bytes. Two registers run the show:

- `%rsp` — stack pointer (top of stack, lowest address).
- `%rbp` — base pointer (bottom of the current frame).

push and pop

`pushpop1.s` demonstrates the two primitives:

```
_start:
    push $10
    pop %rdi

    mov $60,%rax
    syscall
```

`push $10` subtracts 8 from `%rsp` and stores 10 there. `pop %rdi` loads from (`%rsp`) into `%rdi` and adds 8 back. Exit status: 10.

The function prologue and epilogue

To carve out local space:

1. `push %rbp` — save the caller's base pointer.
2. `mov %rsp, %rbp` — new frame starts here.
3. `sub $N, %rsp` — reserve N bytes (multiple of 16 for ABI alignment).

Local i then lives at $-(8*i)(\%rbp)$. To clean up:

1. `mov %rbp, %rsp` — pop all locals at once.
2. `pop %rbp` — restore caller's base pointer.
3. `ret`.

`stackfunk1.s` puts it together:

```

sillyFunk:
    push %rbp
    mov %rsp,%rbp
    sub $16,%rsp
    movq $1,-8(%rbp)
    movq $2,-16(%rbp)
    mov $0,%rax
    add -8(%rbp),%rax
    add -16(%rbp),%rax

    mov %rbp,%rsp
    pop %rbp
    ret

_start:
    call sillyFunk
    mov %rax,%rdi
    mov $60,%rax
    syscall

```

The C analogue is just:

```

long sillyfunk(){
    long num1 = 1;
    long num2 = 2;
    return num1 + num2;
}

```

Exit status is 3.

Linking external functions: readInt and writeInt

`readInt.s` reads a line from `stdin` and parses it as an integer:

```

    .section .bss
buffer: .zero 128

    .section .text
    .global readInt

parseInt:
intRead:
    movb (%rdi,%rcx,1), %r8b
    sub $'0', %r8
    imul $10, %rax
    add %r8, %rax
    inc %rcx
    cmp %rsi, %rcx
    jl intRead
    ret

readInt:
    mov $0, %rax

```

```

    mov $0, %rdi
    lea buffer(%rip), %rsi
    mov $128, %rdx
    syscall

    mov %rax, %rbx
    mov $0, %rcx
    mov $0, %rax
    lea buffer(%rip), %rdi
    dec %rbx
    mov %rbx, %rsi

    call parseInt
    ret

```

Heads up — this is one of our recurring “almost-but-not-quite” bugs. `movb (%rdi, %rcx, 1), %r8b` loads one byte into the low byte of `%r8`, but *doesn’t clear the upper bits*. If `%r8` happened to have garbage in its upper 56 bits, the subsequent `sub` and `add` would compound that garbage. The corrected idiom uses `movzbl` (move byte to long, zero-extended) so the upper bits get zeroed for free. The version in the assembly guide uses `movzbl` for exactly this reason. Worth fixing if you’re using this as a starting point for your own code.

`readInt` also forgets to push/pop `%rbx` around its use — `%rbx` is callee-saved in the System V ABI, so a caller is entitled to find it intact when we return. That’s another fix-it-please.

`writeInt.s` is the mirror image: it takes an integer in `%rdi`, allocates a 32-byte buffer on the stack, fills it *from the end* by repeatedly dividing by 10 and stashing the ASCII digit, then writes the result. It handles negatives, has a special case for `INT_MIN` (which can’t be negated in two’s complement, so we load 2^{63} via `movq` of a 64-bit immediate), and uses `leave` as the shorthand for “`mov %rbp, %rsp; pop %rbp`.”

(There’s also `writeIntBad.s` sitting in the directory — identical to `writeInt.s` except the `.global writeInt` line is commented out. The point: without `.global`, the linker can’t find the symbol from another file, and you get an “undefined reference to `writeInt`” error.)

`echoInt.s` is the smallest possible client:

```

.section .text
.extern readInt,writeInt
.global _start

_start:
    call readInt
    mov %rax,%rdi
    call writeInt

    mov $0,%rdi
    mov $60,%rax
    syscall

```

`.extern` declares that `readInt` and `writeInt` live in another file. Build by assembling each separately and linking together.

Recursion and stack overflow

Every recursive call allocates a fresh stack frame. The Linux default stack is around 4MB — not infinite. `overflowPractice.c` shows what happens when you go too deep:

```
int counter = 0;

int badRec(int n){
    int stuff = 1;
    int moreStuff = 2;
    counter++;
    if(n == 0){
        return 0;
    }
    else{
        printf("%d\n", counter);
        return badRec(n+1)+stuff+moreStuff;
    }
}

int main(){
    badRec(1000);
    return 0;
}
```

Notice the bug: `badRec(n+1)` instead of `badRec(n-1)`. This will never reach the base case — it just keeps allocating frames until the stack collides with the heap and the kernel kills the process. This is the canonical “stack overflow.”

8 Lecture 9 — Processes and the First Look at Threads

A pivot day: assembly roundup, the discussion problem in detail, and then the start of concurrency — processes, fork, and a teaser on threads.

Discussion 4 walkthrough

The discussion was: write a C program that reads two numbers and adds them, then use `gcc -S` to find the addition in the assembly and change it to a multiplication. `discussionDiscussion.c` is the C version:

```
int main(){
    int num1; // -8(%rbp)
    int num2; // -12(%rbp)

    printf("Enter a number: ");
    scanf("%d",&num1);
    printf("Enter another: ");
    scanf("%d",&num2);

    int num3 = num1 + num2; // num3 is -4(%rbp)
    printf("Ta-da! %d\n",num3);
    return 0;
}
```

In `discussionDiscussion.s` the patch is at the addition site — the original was an `addl`, and the discussion modification turns it into:

```
movl -8(%rbp), %edx
movl -12(%rbp), %eax
imul %edx, %eax
movl %eax, -4(%rbp)
```

There's also `dd2.s` where I tried it with the unary form `mull %edx` (which multiplies `%eax` by the operand implicitly — that's the historical x86 form, where the result spans `%edx:%eax`). Both work; `imul` with two operands is cleaner.

Cross-platform notes on calling convention

- Windows on x86-64 has the *same* instruction set — that's set by the chip — but a different calling convention: arguments go in `%rcx`, `%rdx`, `%r8`, `%r9` for the first four, the rest on the stack, and there's a 32-byte “shadow space” the caller has to reserve.
- Modern Macs are on ARM64 (Apple Silicon). The instruction set is totally different, but the calling convention is largely Unix-shaped.

The compiler is what bridges all this — one language, different codegen for each target.

Processes

A *process* is the OS-level unit of “a running program.” The OS bundles together the executable code, register state, file handles, mapped memory, and so on into one process data structure.

A “thread of execution” is the actual entity scheduled onto a CPU core. A process has at least one.

fork: the OG concurrency primitive

`fork1.c` is the simplest possible:

```
#include "stdio.h"
#include "stdlib.h"
#include "unistd.h"

int main(){
    printf("How's it going?\n");

    fork(); // create a new process that's a CLONE of the original

    printf("Hi there\n");
    return 0;
}
```

You'll see “How's it going?” once and “Hi there” twice, because after `fork()` there are two processes and both run the rest of `main`.

`fork2.c` distinguishes parent from child by inspecting the return value of `fork`:

```

int num1 = 10;
pid_t pid = fork();

if(pid != 0){
    num1++;
    printf("I'm the parent. Here's my number %d\n",num1);
}
else{
    num1--;
    printf("I'm the child. Here's my number %d\n",num1);
}

```

fork returns the child's PID in the parent and 0 in the child. The two processes have *independent* copies of `num1` — forking clones the entire memory space, so the parent and child are now operating on different bytes that just happen to have the same name.

`fork3.c` shows what happens when you fork in a loop — you get 2^n processes:

```

int main(){
    for(int i=0; i< 4;i++){
        fork();
        printf("I'm Mr. %d here at index %d\n",getpid(),i);
    }
    return 0;
}

```

After 4 iterations there are 16 processes flooding stdout. This is how runaway forks become fork bombs in real life.

(One thing missing here that we'll fix later: there's no `wait()` in the parent, so children can become zombies until `init` reaps them. Add `wait(NULL);` after `fork()` in real code.)

Threads, briefly

The crucial difference between processes and threads: when you make a new thread, you do *not* clone everything. A thread gets its own registers (including its own `%rip` and stack) but *shares the address space* with all the other threads in the process. That's the source of all the power and all the pain.

`thread1.c` is the minimum:

```

#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    (void)arg;
    printf("Thread executing\n");
    return NULL;
}

int main() {
    pthread_t thread;
    pthread_create(&thread, NULL, thread_function, NULL);
}

```

```

    pthread_join(thread, NULL);
    printf("Thread joined\n");
    return 0;
}

```

`pthread_create` takes a thread handle, an attributes object (NULL for “default”), the function to run, and a `void*` argument to pass to it. `pthread_join` waits for the thread to finish.

`thread2.c` is the sneak preview of why concurrency is hard:

```

int num = 0;

void* thread_function(void* arg) {
    for(int i=0; i<100; i++){
        int num2 = num;
        num2 = num2+1;
        num = num2;
    }
    return NULL;
}

int main() {
    pthread_t thread[100];
    for(int i=0; i<100; i++){
        pthread_create(&thread[i], NULL, thread_function, NULL);
    }
    for(int i=0; i<100; i++){
        pthread_join(thread[i], NULL);
    }
    printf("And the total is: %d\n", num);
    return 0;
}

```

In math-land, that should print 10000 (100 threads each incrementing 100 times). In real life, you’ll often see less, because the read–modify–write of `num` isn’t atomic and two threads can step on each other. That’s a *race condition*, and we’ll spend the next couple of lectures fixing it.

9 Lecture 11 — Function Pointers, Threads in Earnest, and Mutexes

(There was no Lecture 10 directory.) Today: function pointers, because `pthread_create` demands one; `void*` as the universal pointer type; threads with arguments and return values; race conditions; mutexes; and packaging shared state in a struct.

Function pointers

A function pointer is just an address you can call. The syntax is ugly but the idea is simple. `funcpoint1.c` writes a map:

```

int incWrongly(int a){
    return (a + 2);
}

```

```

void map(int arr[], int sz, int (*func)(int)){
    for(int i=0;i<sz;i++){
        arr[i] = func(arr[i]);
    }
}

```

The parameter `int (*func)(int)` is read as “a pointer to a function that takes an `int` and returns an `int`.” You call it just like a regular function: `func(arr[i])`.

`funcpoint2.c` adds a two-argument `reduce`:

```

int reduce(int arr[], int sz,
           int (*func)(int,int),
           int base){
    int acc = base;
    for(int i=0;i<sz;i++){
        acc = func(acc,arr[i]);
    }
    return acc;
}

```

Function pointers are how we get higher-order functions in C — which we need because `pthread_create`’s signature *requires* a function pointer.

void versus void*

`voidArith.c` demonstrates that `void*` arithmetic behaves like `char*` arithmetic — it advances one byte per `+1`:

```

int arr[5];
void* weirdo = arr;
printf("weirdo: %p\n",weirdo);
printf("weirdo+1: %p\n",weirdo+1);

```

This is a GCC extension; standard C says `void*` arithmetic is undefined. But the practical take-away: `void` is “the type of nothing,” and `void*` is “the universal donor of pointer types” — anything can be cast to or from `void*`, which is how C does generic programming.

Threads, with arguments and return values

`threads1.c` is the bare-bones version we already saw last time. `threads2.c` passes an argument:

```

void* func(void* arg){
    printf("Hi I'm a thread and I'm holding: %d\n",
           *((int*)arg));
    return NULL;
}

int main(){
    pthread_t thread;
    int arg = 10;
}

```



```

pthread_create(&thread,NULL,func,&arg);
pthread_join(thread,NULL);
return 0;
}

```

The cast inside the worker — `*((int*)arg)` — is the universal pattern: take the `void*`, lie about its type, dereference. Don't pass pointers to short-lived stack variables across thread boundaries; use heap-allocated or longer-lived storage.

`threads3.c` returns a value:

```

void* func(void* arg){
    int convArg = *((int*)arg);
    int* ret = malloc(sizeof(int));
    *ret = 2*convArg;
    return ret;
}

int main(){
    pthread_t thread;
    int arg = 10;
    int* res;

    pthread_create(&thread,NULL,func,&arg);
    pthread_join(thread,(void**)&res);

    printf("The return value from the thread is: %d\n",*res);
    free(res);
    return 0;
}

```

`pthread_join` takes a `void**` (yes, two stars) so it can write the worker's return value back into a caller-side pointer. The thread `mallocs` its return so it survives past the thread's death; the main thread `free`s it.

Race conditions: `threadRace.c`

```

int counter = 0;

void* incer(void* arg){
    int temp = counter;
    sleep(rand()%3+1); // simulated work
    counter = temp+1;
    return NULL;
}

int main(){
    pthread_t threads[100];
    for(int i=0; i<100;i++){
        pthread_create(threads+i,NULL,incer,NULL);
    }
    for(int i=0; i<100;i++){
        pthread_join(threads[i],NULL);
    }
}

```

```

}
printf("The counter is: %d\n",counter);
return 0;
}

```

The `sleep` is fake-work that makes the race *much* more likely to happen. With 100 threads each doing a non-atomic read-modify-write of `counter`, you'll often see a final value of 1 — everyone read 0, slept, then wrote 1.

Mutexes: threadRace2.c

```

int counter = 0;
pthread_mutex_t mutex;

void* incer(void* arg){
    pthread_mutex_lock(&mutex);

    int temp = counter;
    sleep(rand()%3+1);
    counter = temp+1;
    pthread_mutex_unlock(&mutex);
    return NULL;
}

int main(){
    pthread_t threads[5];
    pthread_mutex_init(&mutex,NULL);

    for(int i=0; i<5;i++){
        pthread_create(threads+i,NULL,incer,NULL);
    }
    for(int i=0; i<5;i++){
        pthread_join(threads[i],NULL);
    }
    printf("The counter is: %d\n",counter);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

A mutex is a thing where exactly one thread can hold the lock at a time — everyone else **locks** and waits. Wrap your critical section between `lock` and `unlock`, and the read-modify-write becomes atomic from the rest of the world's perspective. Now the count is reliably 5.

Bundling state with its lock: threadRace3.c

```

struct guardedCounter {
    int counter;
    pthread_mutex_t mutex;
};

void* incer(void* arg){
    struct guardedCounter* g = (struct guardedCounter*)arg;

```

```

pthread_mutex_lock(&g->mutex);
int temp = g->counter;
sleep(rand()%3+1);
g->counter = temp+1;
pthread_mutex_unlock(&g->mutex);
return NULL;
}

```

The argument is a `void*` that's secretly a `struct guardedCounter*`. Putting the mutex inside the struct means anyone with access to the data also has access to its lock — the protection travels with the resource. This is the shape of nearly all real concurrent code.

For more details and a longer worked discussion of these patterns, see [concurrencyGuide.org](https://concurrencyguide.org) in the repo.

10 File I/O — Files, File Descriptors, argv, and a Line Editor

(This was our deep dive into file I/O, which fits naturally between the threading lectures and the signal lectures since it doesn't involve concurrency itself.)

Everything is a file

In Unix, “file” is a wildly general abstraction. Regular files, directories, devices, terminal emulators, named pipes, and sockets all show up to your program through the same set of operations (`open`, `read`, `write`, `close`). The kernel maintains a per-process table of open file objects, and a *file descriptor* is just an integer index into that table.

File descriptors and open

`fd1.c` is the minimal `open/close`:

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main(){
    char fname[] = "Agenda.org";
    int fd = open(fname,O_RDONLY);

    printf("stdin: %d\n",STDIN_FILENO);
    printf("stdout: %d\n",STDOUT_FILENO);
    printf("stderr: %d\n",STDERR_FILENO);

    if(fd >= 0){
        printf("Hey we opened the file %s and its file descriptor is: %d\n",fname,fd);
    }
    else {
        printf("We couldn't open the file\n");
    }
    close(fd);
    return 0;
}

```

STDIN_FILENO, STDOUT_FILENO, and STDERR_FILENO are the macros for 0, 1, 2 respectively. The first `open` call gets you fd 3 (next free slot).

Command-line arguments: `argv`

`fd3.c` is the canonical `argc/argv` demo:

```
int main(int argc, char* argv[]){
    printf("%d arguments were passed in\n",argc);
    for(int i=0; i<argc; i++){
        printf("The %dth argument is: %s\n",i,argv[i]);
    }
    return 0;
}
```

`argv[0]` is conventionally the program name; the actual arguments start at `argv[1]`. `argc` is the count.

`fd2.c` chains those: it tries to `open` every `argv` entry, prints the resulting fd (or an error), and closes everything at the end. Notice that `argv[0]` is the program name itself and we happily try to `open` it — which works! The binary is just another file.

Reading bytes: `read`

`read1.c` reads up to 1024 bytes from a file:

```
char readData[1024];
int bytesRead = read(fds[1],readData,1024);
printf("We read in %d out of the file %s\n", bytesRead, argv[1]);
```

`read2.c` reads twice in a row, demonstrating that the file's read position advances — the second read picks up where the first left off.

`read3.c` loops until `read` returns 0 (end of file) or negative (error):

```
char readData[10];
int bytesRead = 0;
while((bytesRead = read(fds[1],readData,10)) > 0){
    printf("We read in %d out of the file %s\n", bytesRead, argv[1]);
}
```

This is the canonical “read until done” loop.

Writing our own `cat`

`mycat.c` concatenates all the files named on the command line and writes them to `stdout`:

```
int main(int argc, char* argv[]){
    int numFiles = argc - 1;
    int* fds = malloc(numFiles * sizeof(int));

    for(int i=0; i < numFiles; i++){
        fds[i] = open(argv[i+1],O_RDONLY);
    }
}
```

```

for(int i=0; i < numFiles; i++){
    char readData[1024];
    int bytesRead = 0;
    while((bytesRead = read(fds[i],readData,1024)) > 0){
        write(STDOUT_FILENO,readData,bytesRead);
    }
}
for(int i=0; i< argc; i++){
    close(fds[i]);
}
return 0;
}

```

That’s basically what `cat` does. (Subtle bug in the close loop: it goes from 0 to `argc`, but `fds` is only `numFiles = argc - 1` long. The last `close(fds[argc-1])` reads off the end of the array. Not a crash in practice, but worth fixing.)

The FILE* world: fopen, fgets, fputs

The `stdio` “FILE*” interface is a buffered wrapper over file descriptors. `file1.c`:

```

FILE* ourFile = fopen(argv[1],"r+");

char line[1024];
while( fgets(line,1024,ourFile) != NULL){
    printf("%s",line);
}

```

`fgets` reads a line at a time — much friendlier than raw `read` when you actually have line-structured data.

`weirdcopy.c` uses `fopen` and `fputs` to copy a file line-by-line.

Parsing CSVs: petsDB.c and petsDB2.c

`petsDB.c` parses one line of a comma-separated `name,species,age` file:

```

struct petData {
    char name[50];
    char species[50];
    int age;
};

void readPet(FILE* f, struct petData* p){
    fscanf(f,"%49[^\,],%49[^\,],%d",p->name,p->species,&p->age);
}

```

The `%49[^\,]` format means “read up to 49 chars that aren’t a comma.” `petsDB2.c` extends this into a full loop, using the fact that `fscanf` returns the number of fields it parsed — when that drops below 3 we know we’re done:

```

while(readPet(petFile,&p) == 3){

```

```

    printf("%s is a %s and is %d years old\n",p.name,p.species,p.age);
}

```

Directories with opendir/readdir

directory1.c lists regular files in a directory:

```

DIR* dir = opendir(path);
struct dirent *entry;
while ((entry = readdir(dir)) != NULL) {
    if(entry->d_type == DT_REG){
        printf("%s\n", entry->d_name);
    }
}
closedir(dir);

```

readdir returns one entry per call; NULL means we're done. The d_type field tells you regular file vs. directory vs. symlink vs. socket vs. etc.

openDir.c is similar but listed as part of the same exercise.

File metadata with lstat/fstat

lstatTest.c (and the identical fstatTest.c) builds a tiny version of `ls -l`, decoding the mode bits to print file type and permissions:

```

struct stat file_stat;
lstat(argv[1], &file_stat);

switch (file_stat.st_mode & S_IFMT) {
    case S_IFREG: printf("regular file\n"); break;
    case S_IFDIR: printf("directory\n"); break;
    case S_IFLNK: printf("symbolic link\n"); break;
    // ...
}

printf("%c", file_stat.st_mode & S_IRUSR ? 'r' : '-');
printf("%c", file_stat.st_mode & S_IWUSR ? 'w' : '-');
printf("%c", file_stat.st_mode & S_IXUSR ? 'x' : '-');
// ...

printf("Size of file in bytes: %ld\n", (long) file_stat.st_size);

```

lstat doesn't follow symlinks (it tells you about the link itself), fstat works on an already-open fd, and plain stat follows symlinks. The mode bits are a bitfield — mask with S_IRUSR etc. to read individual permissions.

Putting it together: a line editor

lineEditor.c is the grand finale. It loads a file into a `char**` (an array of lines), then offers an interactive menu to edit, delete, or insert lines, and writes the result back on quit. The interesting bits:

```

void cleanUp(FILE* f, char** ls, int nL){
    ftruncate(fileno(f),0);
    fseek(f, 0, SEEK_SET);
    for(int i=0; i < nL; i++){
        fputs(ls[i],f);
        free(ls[i]);
    }
    free(ls);
    fclose(f);
}

void insLine(int line, char** ls, int* nL){
    char* newLine = malloc(linesize*sizeof(char));
    // shift everything from line..end down by one
    for(int i = *nL; i > line; i--){
        ls[i] = ls[i-1];
    }
    ls[line] = newLine;
    (*nL)++;
}

```

`ftruncate` on the underlying fd zeroes the file; then we `fseek` to the start and write everything back. The line shifting is the classic “insert into an array” move — walk backwards from the end so you don’t overwrite data you haven’t copied yet.

There’s also a tiny demo `cursing.c` for `ncurses` (initialize, print, refresh, wait for keypress, end) — `ncurses` is the standard library for terminal-based UIs and is what you’d build a more polished editor on top of.

11 Lecture 13 — Signals and Signal Handlers

Today: exceptions, processes, virtual memory, and signals — the mechanism by which the OS interrupts your normal control flow.

Exceptions, broadly

The OS interrupts your program in three flavors:

- *Hardware interrupts*: external signals to the CPU, e.g. a key press. Handled by a kernel-maintained interrupt descriptor table.
- *Faults*: things like segfaults (serious) and page faults (often routine — “go fetch this page from RAM”).
- *Traps and aborts*: divide-by-zero, illegal instruction, and similar.

What’s in a process?

Each process bundles together:

- The actual executable code.

- Saved register state (what the kernel snapshots on a context switch).
- Open file descriptors.
- Mapped memory pages.

The *scheduler* is the part of the OS that decides which process gets the CPU next. A *context switch* is the (relatively expensive) operation of saving one process's registers and restoring another's.

Virtual memory gives every process the illusion that it owns the entire address space. The MMU translates virtual addresses to physical ones on the fly, which is how processes are isolated from each other. When working sets exceed RAM, the system starts *swapping* pages to disk and everything grinds to a halt.

getpid and pidLoop.c

pidLoop.c is the simplest possible “find my PID and yell about it” loop:

```
int main(){
    pid_t myPID = getpid();
    while(1){
        printf("I'm mr. %d\n",myPID);
        sleep(1);
    }
    return 0;
}
```

Run it in one terminal, take the PID it prints, then in another terminal you can kill -9 <pid> or kill -USR1 <pid> to send signals at it. That's the setup for everything below.

Common signals

- SIGINT — Ctrl-C
- SIGFPE — divide by zero (and friends)
- SIGKILL — “you're going to die, no questions, no handler”
- SIGSEGV — segfault
- SIGALRM — alarm timer
- SIGSTOP / SIGTSTP — stop / Ctrl-Z
- SIGUSR1, SIGUSR2 — user-defined, no system meaning

Catching SIGINT: uninterruptable.c

```
void uninterruptable(int signum){
    (void)signum;
    write(1, "\nOuch!\n", 6);
}

int main(){
    signal(SIGINT, uninterruptable);
}
```



```

while(1){
    printf("I'm Mr. %d and I cannot be stopped!\n\n",getpid());
    sleep(1);
}
return 0;
}

```

Install a handler with `signal(SIGNAL, handler_function)`; now Ctrl-C prints “Ouch!” instead of killing the process. To actually quit you need a second terminal and `kill -9`, or use `SIGTERM`, etc.

(Note: inside a signal handler, `printf` is not safe — it’s not async-signal-safe and can deadlock if the signal arrived mid-`printf`. Use `write` on the raw fd, as we do here.)

Custom signals: `sigusr1.c` and `sigusr2.c`

`sigusr1.c` listens for two user-defined signals:

```

void sigh_andler(int signum){
    if(signum == SIGUSR1){
        char msg[] = "I guess we're done?\n";
        write(1,msg,strlen(msg));
        exit(0);
    }
    else if(signum == SIGUSR2){
        char msg2[] = "Did you say something?\n";
        write(1,msg2,strlen(msg2));
    }
}

int main(){
    signal(SIGUSR1,sigh_andler);
    signal(SIGUSR2,sigh_andler);

    while(1){
        printf("I'm Mr. %d and I'm just minding my own business...\n",getpid());
        sleep(1);
    }
}

```

From another terminal: `kill -USR1 <pid>` makes it exit; `kill -USR2 <pid>` makes it talk back.

`sigusr2.c` introduces volatile `sig_atomic_t` state shared between handler and main loop:

```

volatile sig_atomic_t state = 0;

void sigh_andler(int signum){
    if(signum == SIGUSR1){
        state +=1;
    }
    else if(signum == SIGUSR2){
        state -= 1;
    }
}

```

```

int main(){
    signal(SIGUSR1,sigh_andler);
    signal(SIGUSR2,sigh_andler);

    while(state < 5){
        printf("I'm Mr. %d and I'm just minding my own business...\n",getpid());
        sleep(1);
    }
}

```

`volatile` tells the compiler “don’t cache this in a register, reload it from memory every time” — otherwise an optimizer might decide `state < 5` can never change inside the loop and turn the whole thing into `while(true)`. `sig_atomic_t` is the type guaranteed to be readable and writable atomically with respect to signals.

Signaling between fork-children: sigusr3.c

```

int main(){
    signal(SIGUSR1,sigh_andler);
    signal(SIGUSR2,sigh_andler);

    pid_t pid = fork();
    if(pid == 0){
        // child: ping the parent every two seconds
        for(int i=0; i<5;i++){
            kill(getppid(), SIGUSR1);
            sleep(2);
        }
    }

    while(state < 5){
        printf("I'm Mr. %d and I'm just minding my own business...\n",getpid());
        sleep(1);
    }
}

```

The child uses `getppid()` (parent PID) and `kill(pid, SIG)` to send `SIGUSR1` to the parent five times. The parent’s handler bumps `state` each time, and once `state >= 5` the parent exits.

An incremental game with signals: forkIncremental.c

```

volatile sig_atomic_t income = 0;
volatile sig_atomic_t gen1 = 0;
volatile sig_atomic_t gen2 = 0;
volatile sig_atomic_t gen3 = 0;

void heartbeat(int signumb){
    printMenu();
    gen2 += gen3;
    gen1 += gen2;
    income += gen1;
}

```

```

int main(){
    pid_t pinger = fork();

    if(pinger == 0){
        while(1){
            pid_t parent = getppid();
            sleep(1);
            kill(parent, SIGUSR1);
        }
    }

    signal(SIGUSR1, heartbeat);

    printMenu();
    while(1){
        int choice;
        scanf("%d", &choice);
        switch(choice){
            case 1: income++; break;
            case 2:
                if(income >= 10){ income -= 10; gen1++; }
                break;
            // ...
        }
    }
}

```

A child process pings the parent once per second with SIGUSR1; the parent's handler advances the game state. Meanwhile the parent's main loop reads keyboard input. This is a real little incremental "cookie-clicker"-style game using nothing but signals as the heartbeat. Next lecture we'll redo this same game with threads instead, which is a much cleaner pattern.

12 Lecture 14 — Threads in Earnest, and Reworking the Incremental Game

The thread review day: re-doing the incremental game with threads and mutexes instead of signals, plus some standalone thread review examples and a teaser on `sigaction`.

Logistics for the assignment

The next coding assignment is parallel processing: do some operation in parallel on a big malloc'd array (hundreds of thousands or millions of elements). Pick a fixed thread count (say 10), and have each thread grab a fixed-size chunk (say 1000 elements) at a time, using a shared counter (protected by a mutex) to track the next chunk. Don't divide the array by the number of threads — the chunk size should be a constant.

Thread review: `threadreview1.c`

```

void* threadWorker(void* arg){
    char* msg = (char*)arg;

```

```

    usleep(rand()%15000);
    printf("%s", msg);
    return NULL;
}

int main(){
    pthread_t thread1, thread2, thread3;

    char msg1[] = "Hi there\n";
    char msg2[] = " how are you?\n";
    char msg3[] = " I hope this message came through\n";

    pthread_create(&thread1, NULL, threadWorker, msg1);
    pthread_create(&thread2, NULL, threadWorker, msg2);
    pthread_create(&thread3, NULL, threadWorker, msg3);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    pthread_join(thread3, NULL);
}

```

Three threads each print a different message after a random short sleep. The output order is unpredictable, which is the point — threads run when the scheduler decides, not when you wrote the `pthread.create`.

(The `usleep` takes microseconds, so `rand()%15000` sleeps up to 15 ms.) After joining all three, we then re-create `thread1` again to show that you can reuse the `pthread_t` variable once the previous use is joined.

Mutex review: threadreview2.c

```

char* msgs[] = {"Hi there ", "how are you? ", "I hope this message came through"};
int counter = 0;
pthread_mutex_t mut;

void* threadWorker(void* arg){
    sleep(rand()%3+1);

    int temp;
    pthread_mutex_lock(&mut);
    temp = counter;
    counter++;
    pthread_mutex_unlock(&mut);
    printf("%s", msgs[temp]);
    return NULL;
}

```

Each thread grabs the lock, snapshots and increments a shared counter, releases the lock, then uses the snapshotted value to pick which message to print. The lock makes the “read-and-increment” indivisible.

Little simulation: test1.c

test1.c is a sketch of an agent simulation:

```
enum lilguyState { eating, sleeping, moving, vibing };

struct lilguyData {
    int id;
    enum lilguyState state;
};

sig_atomic_t simulationRunning = 1;

void handler(int signum){
    char msg[] = "Alright everyone clean up and go!\n";
    simulationRunning = 0;
    write(1,msg,strlen(msg));
}

void* threadWorker(void* arg){
    struct lilguyData dat = *((struct lilguyData*) arg);
    while(simulationRunning){
        printf("I'm Miss %d and I'm ",dat.id);
        printState(dat.state);
        dat.state = rand()%4;
        sleep(rand()%3+1);
    }
    return NULL;
}
```

Five “little guy” threads each randomly cycle through states until SIGINT (Ctrl-C) flips `simulationRunning` to 0 and they all clean up and exit.

Incremental game, take two: threadIncremental.c

```
pthread_mutex_t mut;
int income = 0, gen1 = 0, gen2 = 0, gen3 = 0;

void* heartbeat(void* arg){
    while(1){
        pthread_mutex_lock(&mut);
        printMenu();
        gen2 += gen3;
        gen1 += gen2;
        income += gen1;
        pthread_mutex_unlock(&mut);
        sleep(1);
    }
    return NULL;
}

int main(){
    pthread_t pulse;
    pthread_mutex_init(&mut,NULL);
```

```

pthread_create(&pulse,NULL,heartbeat,NULL);

while(1){
    int choice;
    scanf("%d",&choice);
    switch(choice){
    case 1:
        pthread_mutex_lock(&mut);
        income++;
        pthread_mutex_unlock(&mut);
        break;
        // ...
    }
}
}

```

Same game as last time, but now the heartbeat is a worker thread (not a signal-driven side-effect) and every read or write of the shared state is wrapped in mutex lock/unlock. This is dramatically cleaner than the SIGUSR1-driven version, and is how you'd actually build something like this in production.

Adding a clean shutdown: threadIncremental2.c

```

volatile sig_atomic_t toContinue = 1;

void cleanup(int signum){
    toContinue = 0;
}

// ...
struct sigaction sigsettings;
sigsettings.sa_handler = cleanup;
sigsettings.sa_flags = 0; // makes scanf not block
sigaction(SIGINT,&sigsettings,NULL);

while(toContinue){
    int choice;
    scanf("%d",&choice);
    // ...
}

```

We add a SIGINT handler that flips `toContinue` to 0, and both the heartbeat loop and main loop check that flag. The catch: plain `signal()` sets `SA_RESTART` by default on Linux, which means `scanf` will silently retry after the signal — you'd have to type something to break out. Using `sigaction` with `sa_flags = 0` disables that, so `scanf` returns with an error on signal and the loop checks `toContinue`.

`threadIncremental3.c` and `threadIncremental3.sigaction.c` are tiny variations on the same theme: one with the plain `signal()` call (which will get stuck in `scanf`), one with `sigaction` (which won't), and a version where the cleanup handler also writes a final “make one last purchase” message. The diff between the two versions is the whole pedagogical point: `signal()` is the old, slightly-broken interface; `sigaction()` is what you actually want for real code.

13 Sockets — Networks, the TCP/IP Stack, and Three Echo Servers

This was our networking day. Final project handling has moved to GitHub at <https://github.com/clarissalitt>. The `sockets-lecture/` directory has the working examples.

The TCP/IP model in 5 minutes

- *Application* layer: HTTP, DNS, SSH, BitTorrent — things built on top of the protocols.
- *Transport* layer: TCP (reliable, ordered) and UDP (best-effort, unordered, fast). TCP guarantees every byte arrives and arrives in order; UDP just lobs packets and hopes.
- *Internet* layer: IP addresses (IPv4, 32-bit, four dotted octets; IPv6, 128-bit) and routing.
- *Link* layer: the physical connection — ethernet, wi-fi.

Sockets and network byte order

A *socket* is the OS abstraction for a network endpoint. Surprise: it's just a file descriptor, so once you've opened one you can `read/write` on it like any other file.

Network byte order is *big-endian* (most-significant byte first), which is the opposite of x86-64. So before you stuff a port number or IP address into the socket structures, you call `htons` (host-to-network short) or `htonl` (host-to-network long) to byte-swap.

Echo server, simplest form

`echo_server_simplistic.c` is the bare minimum:

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);

int optval = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR,
           &optval, sizeof(optval));

struct sockaddr_in server_addr;
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
server_addr.sin_port = htons(port);

bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
listen(server_fd, 10);

while (1) {
    struct sockaddr_in client_addr;
    socklen_t client_len = sizeof(client_addr);
    int client_fd = accept(server_fd,
                          (struct sockaddr *)&client_addr,
                          &client_len);

    char buffer[1024];
    ssize_t bytes_received;
```

```

while ((bytes_received = recv(client_fd, buffer,
                             sizeof(buffer), 0)) > 0) {
    send(client_fd, buffer, bytes_received, 0);
}
close(client_fd);
}

```

The five-step recipe:

1. `socket(AF_INET, SOCK_STREAM, 0)` — TCP socket over IPv4.
2. Fill out a `sockaddr_in` (family, IP, port) and `bind` it to the socket.
3. `listen` marks the socket as accepting connections.
4. `accept` blocks until a connection comes in, returning a new fd for that client.
5. `recv` and `send` are the socket flavors of `read` and `write`.

The `SO_REUSEADDR` option lets you restart the server without waiting for the kernel's `TIME_WAIT` timeout, which is a huge quality-of-life thing during development.

You can connect to it with `ncat localhost 8080` (or any port you bound) and watch your typing get echoed back.

Echo server with error handling: `echo_server.c`

`echo_server.c` is the same logic but with proper `perror` calls on every step that can fail, and `inet_ntop` to log the client's IP and port:

```

char client_ip[INET_ADDRSTRLEN];
inet_ntop(AF_INET, &client_addr.sin_addr, client_ip, sizeof(client_ip));
printf("Connection from %s:%d\n", client_ip, ntohs(client_addr.sin_port));

```

Multi-client echo server: `echo_server_threaded.c`

The simplistic version handles exactly one client at a time. To handle many in parallel, we spin up a thread per connection:

```

void *handle_client(void *arg) {
    int client_fd = *(int *)arg;
    free(arg);

    char buffer[1024];
    ssize_t bytes_received;
    while ((bytes_received = recv(client_fd, buffer,
                                sizeof(buffer), 0)) > 0) {
        send(client_fd, buffer, bytes_received, 0);
    }
    close(client_fd);
    return NULL;
}

int main(int argc, char *argv[]) {
    // ... socket/bind/listen as before ...

```



```

while (1) {
    int client_fd = accept(server_fd,
                          (struct sockaddr *)&client_addr,
                          &client_len);

    int *client_fd_ptr = malloc(sizeof(int));
    *client_fd_ptr = client_fd;

    pthread_t thread;
    pthread_create(&thread, NULL, handle_client, client_fd_ptr);
    pthread_detach(thread);
}
}

```

Two things to note. First, we `malloc` a single `int` to pass the `fd` into the thread — if we passed `&client_fd` directly, the next iteration would clobber it before the thread read it. Second, we `pthread_detach` so we don't have to `join` — the thread cleans up after itself when it returns. (We don't have a meaningful return value to wait on.)

The grand finale: `chat_server.c`

A multi-client *broadcast* server: every connected client sees every other client's messages. The data structure:

```

typedef struct {
    int fd;
    int active;
    char ip[INET_ADDRSTRLEN];
    int port;
} Client;

Client clients[MAX_CLIENTS];
pthread_mutex_t clients_mutex = PTHREAD_MUTEX_INITIALIZER;

```

A fixed-size array of clients, protected by a single global mutex. The broadcast routine grabs the lock and writes to every active client except the sender:

```

void broadcast(char *message, int sender_fd) {
    pthread_mutex_lock(&clients_mutex);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (clients[i].active && clients[i].fd != sender_fd) {
            send(clients[i].fd, message, strlen(message), 0);
        }
    }
    pthread_mutex_unlock(&clients_mutex);
}

```

This is everything we've built all term, all in one place: file descriptors, threads, mutexes, structured shared state. Try connecting two terminals to the same server with `ncat` and watch the messages cross-pollinate.

For the deeper details (especially around partial reads, message framing, and what `recv` *actually* returns at TCP boundaries), see [socketsGuide.org](https://socketsguide.org) in the repo.

14 Appendix: Pointer Pyrotechnics (pointerExtras/)

These were extra examples I dropped into the repo for practice — not tied to a specific lecture, but worth a read.

pointymcpointface.c: pointers as a substitute for pass-by-reference

```
void worldsWorstInc(int* p){
    *p = *p + 1;
}

int main(){
    int num = 0;
    printf("%p\n", (void*)&num);
    int* np = &num;

    printf("The variable with address %p has a value of %d\n", (void*)np, *np);

    worldsWorstInc(np);
    worldsWorstInc(np);

    printf("The variable with address %p now has a value of %d\n", (void*)np, *np);
}
```

The two reasons pointers exist in C: dynamic memory (malloc/free), and faking pass-by-reference. This is the latter, distilled.

multiarray.c: dynamic 2D arrays as pointers-to-pointers

```
int** arr = malloc(10*sizeof(int*));
for(int i = 0; i<10; i++){
    arr[i] = malloc(10*sizeof(int));
}

for(int i=0; i < 10; i++){
    for(int j=0; j < 10; j++){
        arr[i][j] = 100*(i+1) + j;
    }
}
```

A 10x10 matrix as an array of 10 pointers, each pointing to a row of 10 ints. The double-subscript `arr[i][j]` expands to `*(*(arr+i)+j)` — two dereferences and two pointer arithmetics. Note this isn't contiguous in memory like a true 2D array (`int arr[10][10]`); the rows can be allocated wherever.

pointstack.c: a linked-list stack with pointer-to-pointer for the head

```
struct Node {
    int v;
    struct Node* next;
};
```

```
void pushStack(struct Node** s, int v){
    struct Node* h = malloc(sizeof(struct Node));
    h->v = v;
    h->next = *s;
    *s = h;
}
```

The function takes `struct Node**` (not `struct Node*`) because it needs to mutate the caller's head pointer. This is the same “pass an address so the function can modify the variable” move from `worldsWorstInc`, just one level of indirection deeper. If you internalize `**` here, every later use of pointer-to-pointer (e.g. `argv`, the `void**` in `pthread_join`) becomes much less mysterious.